Design and Analysis of Algorithms

Department of Computer Science and Engineering

ASSIGNMENT 3

Course Code & Title	24UCYPC303 – Design and Analysis of Algorithms
Unit	Unit III – Dynamic Programming
Targeted CO	COPC303.3
Maximum Marks	40
Date of Submission	24 October 2026
Name PRN	Rushikesh Patil 2124UCSM1037

Q1. Dynamic Programming – General Method and Reliability Design

(a) General method, principle of optimality, and comparison

General method. Dynamic programming (DP) is an algorithm design technique for problems whose solution can be viewed as the result of a *sequence of decisions*. The problem is broken into smaller **overlapping subproblems**; each subproblem is solved only once, its answer is stored in a table, and the answers are combined by a recurrence relation to give the answer to the original problem. The usual steps are:

1. Characterise the structure of an optimal solution (state, stage and decision).
2. Write a recurrence that defines the value of an optimal solution in terms of smaller subproblems (forward form $f_i(x)$ or backward form).
3. Compute the values bottom-up (or top-down with memoisation), storing results in a table / sets such as S^i.
4. Trace back through the table to construct the actual optimal solution (the sequence of decisions).
   Principle of optimality. *In an optimal sequence of decisions, whatever the initial state and the first decision are, the remaining decisions must themselves form an optimal decision sequence with respect to the state that results from the first decision.* In other words, the optimal solution of the whole problem contains optimal solutions of its subproblems. Example: if the shortest path from i to j passes through k, then its i→k and k→j parts are themselves shortest paths. The principle must hold for DP to be applicable; it does not hold, for instance, for the longest *simple* path problem.

Comparison with greedy and divide-and-conquer

Basis	Dynamic programming	Greedy method	Divide and conquer
Idea	Solve all subproblems once, store them and combine them using the principle of optimality	Make the locally best choice at each step and never reconsider it	Split into independent subproblems, solve recursively and combine
Subproblems	Overlapping (the same subproblem occurs many times)	One subproblem remains after each choice	Independent / nonoverlapping
Decision sequences	Many sequences are considered, but those that cannot be optimal are discarded	Only one decision sequence is generated	Not a decision-sequence method

Optimality	Always gives an optimal solution (when the principle of optimality holds)	Optimal only for problems with the greedy-choice property (e.g. fractional knapsack, MST)	Depends on the problem (not an optimisation technique as such)
Table / storage	Uses a table or sets to store results	No table	No table (results of recursive calls are recomputed)

Approach	Bottom-up (or memoised topdown)	Top-down, one step at a time	Top-down recursion
Examples	0/1 knapsack, Floyd–Warshall, TSP, reliability design	Fractional knapsack, Prim, Kruskal, Huffman	Merge sort, quick sort, binary search

(b) Reliability design (series system, budget 105)

Problem. Maximise the system reliability $\Pi\ \phi_i(m_i)$ subject to $\Sigma\ c_i m_i \le 105$, with $1 \le m_i \le u_i$, where m_i is the number of copies (in parallel) of device D_i. With m_i copies the stage reliability is $\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$.

Upper bounds on copies. Every device must be used at least once, which costs 30 + 15 + 20 = 65, leaving 105 − 65 = 40 for extra copies. Hence $u_i = \lfloor (c - \Sigma c_j + c_i) / c_i \rfloor$:

Device	c_i	r_i	u_i	$\phi_i(1)$	$\phi_i(2)$	$\phi_i(3)$
D1	30	0.9	⌊(40+30)/30⌋ = 2	0.9	0.99	–
D2	15	0.8	⌊(40+15)/15⌋ = 3	0.8	0.96	0.992
D3	20	0.5	⌊(40+20)/20⌋ = 3	0.5	0.75	0.875

Method. S^i is the set of (reliability, cost) pairs obtainable with devices D1…D_i; $S^o = \{(1, 0)\}$. S^i_j (j copies of D_i) is obtained from S^{i-1} by $(R, C) \rightarrow (R \cdot \phi_i(j),\ C + j \cdot c_i)$. $S^i = \cup_j S^i_j$ after two clean-ups: (1) **infeasible pairs** are dropped – a pair is kept only if C plus the cost of one copy of every remaining device is ≤ 105; (2) **dominance (purging) rule** – a pair (R_1, C_1) is deleted if another pair (R_2, C_2) has $R_2 \ge R_1$ and $C_2 \le C_1$.

$S^o = \{(1, 0)\}$

Stage 1 (D1). Costs of the remaining devices = 15 + 20 = 35, so cost ≤ 70 is required.

Set	Pairs (R, C)
S^1_1 (1 copy)	(0.9, 30)
S^1_2 (2 copies)	(0.99, 60)
S^1	**{(0.9, 30), (0.99, 60)}** – no pair is dominated

Stage 2 (D2). Remaining device D3 costs 20, so cost ≤ 85 is required.

Set	Pairs (R, C)	Remarks
S^2_1 (1 copy, ϕ = 0.8)	(0.72, 45), (0.792, 75)	0.9×0.8 = 0.72, 30+15 = 45; 0.99×0.8 = 0.792, 60+15 = 75
S^2_2 (2 copies, ϕ = 0.96)	(0.864, 60)	(0.99×0.96 = 0.9504, 90) is dropped: 90 + 20 > 105
S^2_3 (3 copies, ϕ = 0.992)	(0.8928, 75)	(0.99×0.992 = 0.98208, 105) is dropped: 105 + 20 > 105
Merged, sorted by cost	(0.72,45), (0.864,60), (0.792,75), (0.8928,75)	(0.792, 75) is dominated by (0.864, 60) and (0.8928, 75) → purged
S^2	**{(0.72, 45), (0.864, 60), (0.8928, 75)}**	

Stage 3 (D3). No devices remain, so only cost ≤ 105 is required.

Set	Pairs (R, C)	Remarks
S^3_1 (1 copy, ϕ = 0.5)	(0.36, 65), (0.432, 80), (0.4464, 95)	each pair of S^2 × 0.5, cost + 20
S^3_2 (2 copies, ϕ = 0.75)	(0.54, 85), (0.648, 100)	(0.6696, 115) is dropped: cost > 105
S^3_3 (3 copies, ϕ = 0.875)	(0.63, 105)	(0.756, 120), (0.7812, 135) are dropped
Merged, sorted by cost	(0.36,65), (0.432,80), (0.54,85), (0.4464,95), (0.648,100), (0.63,105)	(0.4464, 95) is dominated by (0.54, 85); (0.63, 105) is dominated by (0.648, 100) → purged
S^3	**{(0.36, 65), (0.432, 80), (0.54, 85), (0.648, 100)}**	

Optimal pair. The pair with the maximum reliability in S^3 is **(0.648, 100)**, and 100 ≤ 105.

Trace back. (0.648, 100) ∈ S^3_2, so D3 has **2 copies**; it came from (0.648 / 0.75, 100 − 40) = (0.864, 60) ∈ S^2_2, so D2 has **2 copies**; this came from (0.864 / 0.96, 60 − 30) = (0.9, 30) ∈ S^1_1, so D1 has **1 copy**.

Answer: (m_1, m_2, m_3) = **(1, 2, 2)**, i.e. one copy of D1, two of D2 and two of D3. Cost = 30 + 2×15 + 2×20 = 100 ≤ 105. Maximum reliability = 0.9 × 0.96 × 0.75 = **0.648**. (Confirmed by checking all 3×3×3 allocations that fit in the budget.)

Q2. 0/1 Knapsack Problem using Dynamic Programming

n = 4, P = (3, 4, 5, 6), W = (2, 3, 4, 5), M = 8. Maximise $\Sigma\, p_i x_i$ subject to $\Sigma\, w_i x_i \le M$, $x_i \in \{0, 1\}$.

Recurrence relation. Let $f_i(y)$ be the maximum profit using items 1…i with capacity y:

$$f_i(y) = \max \{ f_{i-1}(y),\ f_{i-1}(y - w_i) + p_i \},\ f_o(y) = 0 \text{ for } y \ge 0,\ f_i(y) = -\infty \text{ for } y < 0.$$

In the set form, S^i is the set of (profit, weight) pairs that are obtainable from the first i items: $S^o = \{(0, 0)\}$, $S^i = S^{i-1} \cup S^i_1$, where $S^i_1 = \{(P + p_i,\ W + w_i) : (P, W) \in S^{i-1}\}$. Pairs with weight > M are discarded. **Purging (dominance) rule:** if S^i contains (P_j, W_j) and (P_k, W_k) with $P_j \le P_k$ and $W_j \ge W_k$, then (P_j, W_j) is removed.

i	Item (p , w)		= + (p , w)	after merge and purge
0	–	–	–	$S^o = \{(0,0)\}$
1	(3, 2)	{(0,0)}	{(3,2)}	$S^1 = \{(0,0), (3,2)\}$
2	(4, 3)	S^1	{(4,3), (7,5)}	$S^2 = \{(0,0), (3,2), (4,3), (7,5)\}$
3	(5, 4)	S^2	{(5,4), (8,6), (9,7), (12,9)✗}	$S^3 = \{(0,0), (3,2), (4,3), (5,4), (7,5), (8,6), (9,7)\}$
4	(6, 5)	S^3	{(6,5), (9,7), (10,8), (11,9)✗, (13,10)✗, (14,11)✗, (15,12)✗}	$S^4 = \{(0,0), (3,2), (4,3), (5,4), (7,5), (8,6), (9,7), (10,8)\}$

Details of the purging.

- S^1, S^2: no pair is dominated (profit and weight both increase together).
- S^3: (12, 9) exceeds M = 8, so it is deleted. No other pair is dominated.
- S^4: after merging S^3 with $S^4_1 = \{(6,5), (9,7), (10,8)\}$ the pair **(6, 5) is purged** because (7, 5) has a larger profit for the same weight; the duplicate (9, 7) is kept once. The pairs (11,9), (13,10), (14,11), (15,12) have weight > 8 and are discarded.

Optimal profit. The last pair of S^4 with weight ≤ 8 is (10, 8). So the maximum profit is **= 10**.

Trace back (finding x). A pair (P, W) in S^i has $x_i = 1$ iff it is **not** in S^{i-1} (then its predecessor is $(P - p_i, W - w_i)$); otherwise $x_i = 0$.

Step	Pair	In	Decision	Next pair
i = 4	(10, 8)	$(10,8) \notin S^3$	$x_4 = 1$	(10−6, 8−5) = (4, 3)
i = 3	(4, 3)	$(4,3) \in S^2$	$x_3 = 0$	(4, 3)
i = 2	(4, 3)	$(4,3) \notin S^1$	$x_2 = 1$	(4−4, 3−3) = (0, 0)
i = 1	(0, 0)	$(0,0) \in S^0$	$x_1 = 0$	(0, 0)

Answer: (x_1, x_2, x_3, x_4) = **(0, 1, 0, 1)** – items 2 and 4 are selected. Profit = 4 + 6 = **10**, weight = 3 + 5 = **8 = M**.

Q3. All-Pairs Shortest Path Floyd's Algorithm

Idea. Let $A^k(i, j)$ be the length of a shortest path from i to j that uses only vertices {1,…,k} as intermediate vertices. Either the path does not use vertex k (length $A^{k-1}(i, j)$) or it goes i → k → j through k (length $A^{k-1}(i, k) + A^{k-1}(k, j)$). By the principle of optimality:

$$A^k(i, j) = \min \{ A^{k-1}(i, j),\ A^{k-1}(i, k) + A^{k-1}(k, j) \},\ A^o(i, j) = \text{cost}(i, j),\ A^o(i, i) = 0.$$

```
AlgorithmAllPairs(cost,A,n)
//cost[1:n,1:n]isthecostmatrix;A[i,j]willbethelengthofthe
//shortestpathfromitoj.
{ fori:=1tondo
    forj:= 1tondo
            A[i,j] := cost[i,j];                //A0
    for k := 1 to n do
        fori:= 1tondo
        forj:=1tondo
                A[i,j]:=min(A[i,j],A[i,k]+A[k,j]);
}
```

The same matrix A can be updated in place: during iteration k, row k and column k do not change, because $A^k(k, j) = \min\{A^{k-1}(k, j), A^{k-1}(k, k) + A^{k-1}(k, j)\} = A^{k-1}(k, j)$ as $A^{k-1}(k, k) = 0$ (and similarly for column k).

the given cost matrix (direct edges only)

	1	**2**	**3**	**4**
1	0	3	∞	7
2	8	0	2	∞
3	5	∞	0	1
4	2	∞	∞	0

A^1 intermediate vertex 1 allowed

	1	**2**	**3**	**4**
1	0	3	∞	7
2	8	0	2	**15**
3	5	**8**	0	1
4	2	**5**	∞	0

Every entry is compared with $A^0(i,1) + A^0(1,j)$. Improvements: (2,4): 8 + 7 = **15** < ∞; (3,2): 5 + 3 = **8** < ∞; (4,2): 2 + 3 = **5** < ∞. Other entries do not improve, e.g. (3,4): 5 + 7 = 12 > 1. **A^2 intermediate vertices {1, 2} allowed**

	1	**2**	**3**	**4**
1	0	3	**5**	7
2	8	0	2	15
3	5	8	0	1
4	2	5	**7**	0

Using $A^1(i,2) + A^1(2,j)$: (1,3): 3 + 2 = **5** < ∞; (4,3): 5 + 2 = **7** < ∞. No other entry improves, e.g. (1,4): 3 + 15 = 18 > 7.

A^3 intermediate vertices {1, 2, 3} allowed

	1	**2**	**3**	**4**

1	0	3	5	**6**
2	**7**	0	2	**3**
3	5	8	0	1
4	2	5	7	0

Using $A^2(i,3) + A^2(3,j)$: (1,4): 5 + 1 = **6** < 7; (2,1): 2 + 5 = **7** < 8; (2,4): 2 + 1 = **3** < 15. No other entry improves, e.g. (4,1): 7 + 5 = 12 > 2. **intermediate vertices {1, 2, 3, 4} allowed (final answer)**

	1	**2**	**3**	**4**
1	0	3	5	6
2	**5**	0	2	3
3	**3**	**6**	0	1
4	2	5	7	0

Using $A^3(i,4) + A^3(4,j)$: (2,1): 3 + 2 = **5** < 7; (3,1): 1 + 2 = **3** < 5; (3,2): 1 + 5 = **6** < 8. No other entry improves, e.g. (1,3): 6 + 7 = 13 > 5.

Result: A^4 gives the shortest distance between every pair of vertices. For example, the shortest path 3 → 2 costs 6 (3 → 4 → 1 → 2 = 1 + 2 + 3), and 2 → 1 costs 5 (2 → 3 → 4 → 1 = 2 + 1 + 2).

Complexity.

Time: the statement A[i,j] := min(A[i,j], A[i,k] + A[k,j]) takes O(1) time and is executed once for every triple (k, i, j) of the three nested loops, i.e. n × n × n times. $T(n) = n^3 + n^2$ (initialisation) = , in the best, average and worst cases. (Here n = 4 → $4^3 = 64$ comparisons.)

Space: only the n × n matrix A is needed because the update can be done in place, so the space is . (Storing all of $A^0 \ldots A^n$ separately, as done above for explanation, would need $\Theta(n^3)$.)

Q4. Travelling Salesperson Problem using Dynamic Programming

(a) Recurrence and optimal tour

Recurrence. Let g(i, S) be the length of a shortest path that starts at vertex i, goes through every vertex in the set S exactly once and ends at vertex 1 (i S, 1 S). Then

$$g(i,) = c_{i1}, \qquad g(i, S) = \min \text{ over } j \in S \text{ of } \{ c_{ij} + g(j, S - \{j\}) \},$$

and the optimal tour length is $g(1, V - \{1\}) = \min \text{ over } 2 \le k \le n \text{ of } \{ c_{1k} + g(k, V - \{1, k\}) \}$.

```
fori:=2tondog(i,{}):=c[i,1];
fork:=1ton-2do     //sizeofsetS foreachSsubsetof{2..n}with|S|=kdo
    foreachinotinSandi!=1do g(i,S) := min over j in S of ( c[i,j] + g(j, S-{j}) );
    //rememberbestj
g(1,{2..n}) := min over k of ( c[1,k] + g(k, {2..n}-{k}) );
```

Cost matrix:

	1	**2**	**3**	**4**
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12

4	8	8	9	0

|S| = 0: $g(i,) = c_{i1}$

g(2,)	g(3,)	g(4,)
$c_{21} = 5$	$c_{31} = 6$	$c_{41} = 8$

|S| = 1: $g(i, \{j\}) = c_{ij} + g(j,)$

g(i, S)	Computation	Value
g(2, {3})	$c_{23} + g(3,) = 9 + 6$	15
g(2, {4})	$c_{24} + g(4,) = 10 + 8$	18
g(3, {2})	$c_{32} + g(2,) = 13 + 5$	18
g(3, {4})	$c_{34} + g(4,) = 12 + 8$	20
g(4, {2})	$c_{42} + g(2,) = 8 + 5$	13
g(4, {3})	$c_{43} + g(3,) = 9 + 6$	15

|S| = 2:

g(i, S)	Computation (minimum over j S)	Value	Best j
g(2, {3,4})	min{ $c_{23} + g(3,\{4\}) = 9 + 20 = 29$, $c_{24} + g(4,\{3\}) = 10 + 15 = 25$ }	25	4
g(3, {2,4})	min{ $c_{32} + g(2,\{4\}) = 13 + 18 = 31$, $c_{34} + g(4,\{2\}) = 12 + 13 = 25$ }	25	4
g(4, {2,3})	min{ $c_{42} + g(2,\{3\}) = 8 + 15 = 23$, $c_{43} + g(3,\{2\}) = 9 + 18 = 27$ }	23	2

|S| = 3 (final):

g(1, {2,3,4})	Computation (minimum over k)	Value
via k = 2	$c_{12} + g(2,\{3,4\}) = 10 + 25$	35
via k = 3	$c_{13} + g(3,\{2,4\}) = 15 + 25$	40
via k = 4	$c_{14} + g(4,\{2,3\}) = 20 + 23$	43
g(1,{2,3,4})	**minimum, obtained with k = 2**	**35**

Tour reconstruction. From g(1,{2,3,4}) the best choice is 2 → go to vertex 2; from g(2,{3,4}) the best choice is 4 → vertex 4; from g(4,{3}) the only choice is 3 → vertex 3; then return to 1.

Optimal tour: 1 2 4 1, with cost 10 + 10 + 9 + 6 = **35**.

Check by enumeration: 1-2-3-4-1 = 39; **1-2-4-3-1 = 35**; 1-3-2-4-1 = 46; 1-3-4-2-1 = 40; 1-4-2-3-1 = 47; 1-4-3-2-1 = 47. The minimum is 35, which agrees with the DP result.

(b) Time complexity and practicality

Number of subproblems. The values g(i, S) are needed for every i ≠ 1 and every S V − {1, i}. For |S| = k there are (n − 1)·C(n − 2, k) such values (for n = 4 this gives 3 + 6 + 3 = 12, plus the final g(1, V−{1}) = 13 values as computed above).

Time. Computing a value g(i, S) with |S| = k needs k additions and k − 1 comparisons, i.e. O(k) work. The total is

$$\Sigma\ (k = 1 \text{ to } n-2)\ k \cdot (n-1) \cdot C(n-2, k) = (n-1)(n-2) \cdot 2^{n-3} = \mathbf{O(n^2 \cdot}$$

(using $\Sigma\, k \cdot C(m, k) = m \cdot 2^{m-1}$ with $m = n - 2$). The final step adds only O(n). **Space:** all values of the previous level must be stored, so $O(n \cdot 2^n)$ space (at most $n \cdot 2^{n-2}$ entries for the whole table).

Comparison with brute force. Trying all (n − 1)! tours takes O(n!) time; DP reduces this to $O(n^2 2^n)$, which is a large improvement, but it is still exponential.

n	(n 1)! tours	operations	table entries
10	362,880	$\approx 1.0 \times 10^5$	$\approx 1.0 \times 10^4$
20	$\approx 1.2 \times 10^{17}$	$\approx 4.2 \times 10^8$	$\approx 2.1 \times 10^7$
30	$\approx 8.8 \times 10^{30}$	$\approx 9.7 \times 10^{11}$	$\approx 3.2 \times 10^{10}$
40	$\approx 2.0 \times 10^{46}$	$\approx 1.8 \times 10^{15}$	$\approx 4.4 \times 10^{12}$

Why it is not practical for large n:

The running time grows by a factor of about 2 for every extra city, so it doubles with each additional vertex: n = 30 already needs about 10^{12} operations.

The memory requirement $O(n \cdot 2^n)$ is usually the first limit: for n = 30 the table has about 3×10^{10} entries (over 100 GB at 4 bytes each), which cannot be held in main memory.

TSP is NP-hard, and no polynomial-time exact algorithm is known; DP is therefore useful only for small instances (roughly $n \leq 20$–25). For larger instances, branch-and-bound, approximation algorithms or heuristics (nearest neighbour, 2-opt, Christofides) are used instead.

— End of Assignment —